

Guia 1 — Git Básico + Configurações

Este guia ensina Git **do zero**, com explicações claras, exemplos reais e saídas do terminal comentadas. Ideal para quem está começando e quer entender **o que está fazendo** em cada comando.

1. O que é Git (em poucas palavras)

Git é um **sistema de controle de versão distribuído**.

Ele registra o histórico completo do seu projeto e permite voltar no tempo, comparar versões e trabalhar com segurança — tudo isso **sem depender de internet**.

Quando você usa Git em uma pasta, ele cria uma pasta oculta chamada:

```
.git
```

Essa pasta transforma uma pasta comum em um **repositório Git** e guarda:

- histórico de commits
- branches
- configurações
- referências internas
- metadados
- objetos compactados do Git

Nunca mexa manualmente dentro da pasta `.git`.

Ela é o “cérebro” do repositório — alterar algo ali pode corromper todo o histórico.

Git funciona registrando **snapshots** (fotos) do estado do projeto a cada commit.

Ele não salva arquivos inteiros repetidamente — salva **apenas as diferenças**, o que o torna rápido e eficiente.

Analogia:

Pense no Git como um “histórico infinito” do seu projeto, onde cada commit é uma foto do momento.

2. Iniciando um repositório — `git init`

Para começar a versionar uma pasta:

```
git init
```

Saída típica:

```
Initialized empty Git repository in C:/Users/ander/exemplo/.git/
```

Isso significa:

- Git criou um repositório vazio
- A pasta `.git` foi criada

Verificando:

```
ls -a
```

```
.  .. .git arquivo1.py
```

Quando usar `git init`?

- quando você cria um projeto do zero

Quando **não** usar?

- ao clonar um repositório (o Git já cria `.git` automaticamente)



3. Configurações do Git — system, global e local

Git possui **3 níveis de configuração**, cada um armazenado em um arquivo diferente:

Nível	Afeta quem?	Arquivo onde fica
system	todos os usuários da máquina	<code>/etc/gitconfig</code>
global	apenas o usuário atual	<code>~/.gitconfig</code>
local	apenas o repositório atual	<code>.git/config</code>

✓ Ordem de prioridade

```
local > global > system
```

Se uma configuração existe no nível local, ela sobrescreve a global e a system.



3.1. Ver configurações

Ver todas as configurações ativas

```
git config --list
```

Ver apenas as globais

```
git config --global --list
```

Ver apenas as locais

```
git config --local --list
```

Ver de onde vem cada configuração

```
git config --show-origin --list
```

Exemplo:

```
file:C:/Users/ander/.gitconfig  user.name=Anderson  
file:.git/config               user.email=local@exemplo.com
```



3.2. Definir nome e e-mail

Git precisa saber **quem está fazendo os commits**.

Global (recomendado)

```
git config --global user.name "Seu Nome"  
git config --global user.email "seu-email@exemplo.com"
```

Local (somente no repositório atual)

```
git config user.name "Nome Local"  
git config user.email "email-local@exemplo.com"
```

Ver origem das configurações

```
git config --show-origin --list
```

Exemplo:

```
file:C:/Users/ander/.gitconfig  user.name=Anderson  
file:.git/config               user.email=local@exemplo.com
```

✖ 3.3. Remover (unset) configurações

Local

```
git config --unset user.name  
git config --unset user.email
```

Global

```
git config --global --unset user.name  
git config --global --unset user.email
```

System (raro, exige permissões elevadas)

```
git config --system --unset user.name  
git config --system --unset user.email
```

✚ 3.4. Outras configurações úteis

Branch padrão como `main`

```
git config --global init.defaultBranch main
```

Editor padrão (VS Code)

```
git config --global core.editor "code --wait"
```

Ativar cores

```
git config --global color.ui auto
```

`.gitignore` global

```
git config --global core.excludesfile ~/.gitignore_global
```

📌 Observação importante sobre `git config` e `git init`

✓ Você **pode** rodar `git config` antes de `git init`.

Isso porque:

- `git config --global` escreve no arquivo `~/.gitconfig`
- esse arquivo **não depende** de existir um repositório
- ele vale para **todos os repositórios futuros**

Ou seja:

configurações globais não precisam de um repositório Git para existir.

4. Ver o estado do repositório — `git status`

```
git status
```

Exemplo:

```
On branch main
No commits yet
Untracked files:
  script.py
```

Significa:

- você está na branch `main`
- não há commits
- `script.py` não está sendo rastreado

Outros estados importantes:

- **Changes not staged for commit** → arquivo modificado, mas não adicionado
 - **Changes to be committed** → arquivo na staging area
 - **working tree clean** → nada a fazer
-



5. Adicionar arquivos — `git add`

Adicionar um arquivo:

```
git add script.py
```

Adicionar todos:

```
git add .
```

Verificando:

```
git status
```

```
Changes to be committed:  
new file: script.py
```

A *staging area* é como uma bandeja de supermercado:

- você coloca itens nela (`git add`)
- passa tudo no caixa (`git commit`)



6. Salvar uma versão — `git commit`

```
git commit -m "Mensagem clara"
```

Exemplo:

```
[main (root-commit) 1a2b3c4] Adiciona script inicial  
1 file changed, 10 insertions(+)  
create mode 100644 script.py
```

Significa:

- **root-commit** → primeiro commit
- **1a2b3c4** → hash do commit

- **10 insertions** → 10 linhas adicionadas

Boas práticas:

- mensagens claras
- verbos no imperativo
- explique o que foi feito

7. **.gitignore** — ignorando arquivos corretamente

O arquivo **.gitignore** diz ao Git **quais arquivos e pastas devem ser ignorados** — ou seja, não rastreados, não adicionados, não commitados e não enviados ao repositório remoto.

Ele é essencial para manter o repositório **limpo, seguro e leve**.

7.1. Por que usar um **.gitignore**?

Você deve ignorar arquivos que:

- **não fazem parte do código-fonte**
- **são gerados automaticamente**
- **contêm informações sensíveis**
- **são específicos da sua máquina**
- **não devem ir para o GitHub**

Exemplos comuns:

- pastas de cache (**__pycache__**/, **.pytest_cache**/)
- ambientes virtuais (**venv**/)
- arquivos de configuração local (**.env**)
- logs (***.log**)
- arquivos temporários (***.tmp**)
- dependências reinstaláveis (**node_modules**/)

7.2. Onde criar o **.gitignore**?

Crie o arquivo **na raiz do repositório**, no mesmo nível da pasta **.git**:

```
touch .gitignore
```

Estrutura típica:

```
meu-projeto/  
├── .git/  
├── .gitignore  
├── src/  
└── README.md
```

7.3. O que escrever dentro do `.gitignore`?

Você escreve padrões de arquivos ou pastas que o Git deve ignorar.

Exemplo básico:

```
__pycache__/  
.env  
*.log
```

Explicando:

- **pycache/** → ignora a pasta inteira
- **.env** → ignora o arquivo .env
- ***.log** → ignora todos os arquivos .log

7.4. Ignorando arquivos que já estão sendo rastreados

Se você já comitou um arquivo e depois o adicionou ao `.gitignore`, ele não será ignorado automaticamente.

Você precisa removê-lo do Git (mas não do disco):

```
git rm --cached arquivo.txt
```

Depois disso, o Git passa a ignorá-lo.

7.5. `.gitignore` global (para todos os repositórios)

Você pode criar um `.gitignore` que vale para todos os seus projetos:

```
git config --global core.excludesfile ~/.gitignore_global
```

Exemplo de `.gitignore_global`:


```
# Arquivos do sistema
Thumbs.db
.DS_Store

# Configurações de editor
.vscode/
.idea/
```

7.6. Dicas profissionais

- Sempre ignore arquivos sensíveis (.env, chaves, tokens).
- Nunca coloque senhas no GitHub.
- Evite ignorar arquivos importantes por engano.
- Use `.gitignore` para manter o repositório limpo e leve.

Você pode criar seu `.gitignore` de duas formas:

1) Copiar um modelo pronto:

<https://github.com/github/gitignore>

OU

2) Gerar um `.gitignore` personalizado:

gitignore.io

8. Ver histórico — `git log`

O comando `git log` mostra o **histórico de commits** do repositório, incluindo:

- hash do commit
- autor
- data
- mensagem do commit

```
git log
```

Exemplo:

```
commit 1a2b3c4
Author: Anderson
Date: Tue Jan 1

    Adiciona script inicial
```

📌 8.1. Versão resumida

Mostra cada commit em uma linha:

```
git log --oneline
```

Saída típica:

```
1a2b3c4 Adiciona script inicial
```

📌 8.2. Navegação dentro do log

Quando o log abre, ele usa um pager (como o less).

Comandos úteis:

- ↑ / ↓ → rolar
- space → próxima página
- b → página anterior
- q → sair

📌 8.3. Mostrar gráfico das branches

Visualização muito útil para entender merges e branches:

```
git log --oneline --graph --decorate --all
```

Exemplo:

```
* 1a2b3c4 (HEAD -> main) Adiciona script inicial
* 9f8e7d6 (feature/login) Cria tela de login
```

📌 8.4. Limitar quantidade de commits

Mostrar apenas os últimos 5 commits:

```
git log -5
```

📌 8.5. Filtrar por autor

```
git log --author="Anderson"
```

✂ 8.6. Filtrar por palavra na mensagem

```
git log --grep="login"
```

✂ 8.7. Mostrar alterações junto com o commit

```
git log -p
```

Mostra o patch (as diferenças) de cada commit.

✂ 8.8. Mostrar apenas nomes de arquivos alterados

```
git log --name-only
```

✂ 8.9. Mostrar estatísticas resumidas

```
git log --stat
```

Exemplo:

```
script.py | 10 ++++++++  
1 file changed, 10 insertions(+)
```

✂ 8.10. Combinação útil para o dia a dia

```
git log --oneline --decorate --graph --all
```

Essa é a visualização preferida de muitos desenvolvedores, pois mostra:

- commits resumidos
- branches
- merges
- HEAD
- tags

- estrutura visual do histórico



9. Fluxo básico de trabalho (local)

```
git init
git config --global user.name "Nome"
git config --global user.email "Email"
git status
git add .
# Obs.: é necessário criar ou modificar um arquivo para enviá-lo ao staging area
git commit -m "mensagem"
git log --oneline
```



Fim do Guia 1 — Git Básico + Configurações

Agora você entende:

- como iniciar um repositório
- como configurar o Git
- como interpretar saídas do terminal
- como adicionar arquivos
- como fazer commits
- como ignorar arquivos
- como ver o histórico
- como seguir o fluxo básico de trabalho

Esse é o alicerce para todos os próximos guias.